HAQMS Internship Assignment Documentation

Overview

This submission focuses on improving the existing HAQMS full-stack hospital management application across security, performance, backend correctness, frontend stability, and incomplete feature coverage.

The goal was not to rewrite the entire application, but to prioritize the highest-impact issues, fix critical bugs, improve code quality, and make the application more reliable and production-ready.

Approach

I approached the assignment in the following order:

1. Set up the project locally and run the database with seeded data.

2. Explore the main user flows for Admin, Doctor, Receptionist, and Queue.

3. Identify high-risk issues first, especially security and authorization problems.

4. Fix backend correctness and performance bottlenecks.

5. Fix frontend crashes and broken UI flows.

6. Complete the missing patient history feature.

7. Re-test the critical doctor and patient workflows.

This prioritization was intentional. I focused first on issues that could cause data leaks, unauthorized access, broken workflows, or scalability problems.

Issues Identified and Fixes Implemented

1. Security Fixes

- Removed plaintext password logging from the authentication flow.
- Reduced JWT expiry from `365d` to `24h`.
- Removed unsafe JWT `ignoreExpiration` behavior.
- Fixed SQL injection risk in `doctors.js` by replacing raw unsafe queries with Prisma ORM queries.
- Fixed admin authorization so admin-only behavior actually enforces the `ADMIN` role.
- Removed password hash exposure from API responses.

2. Backend and Database Improvements

- Created and finalized the Prisma schema for:
- `User`
- `Patient`
- `Doctor`
- `Appointment`
- `QueueToken`
- Added proper indexes and constraints.
- Ran migrations and seeded the database.
- Fixed N+1 query behavior in the appointments flow by using Prisma `include`.
- Fixed sequential doctor stats queries by replacing serial awaits with `Promise.all`.
- Fixed in-memory patient pagination by moving filtering and pagination to the database layer using `take` and `skip`.
- Improved the reports endpoint by replacing slower nested processing with aggregation logic.

3. Frontend Fixes

- Fixed a memory leak in the queue page by cleaning up `setInterval`.
- Fixed a null crash caused by `medicalHistory.toUpperCase()` on nullable values.
- Fixed a hooks order issue by moving the early unauthenticated return after hooks.
- Updated unsafe `user.role` accesses to `user?.role` where needed.
- Fixed login response handling to match the backend response shape (`data.data.token`).
- Fixed doctors endpoint response handling to match the backend shape.
- Made the queue listing endpoint public as required for the queue screen.
- Added missing `.gitignore` coverage for:
- `node_modules`
- `.env` and `.env.*`
- frontend build artifacts

4. Incomplete Feature Completion

- Created the missing patient history page at:
- `frontend/src/app/patients/[id]/history-records/page.js`
- Verified that the page loads patient details and appointment records.

5. Doctor Dashboard Bug Fix

One important remaining functional issue in the app was the doctor dashboard not showing seeded appointments for the logged-in doctor.

Root cause

The dashboard tried to match the authenticated `User` record to a `Doctor` record using `userId`, but the `Doctor` model did not actually contain a `userId` relation in the schema.

Fix applied

- Updated the doctor dashboard logic so it safely resolves the doctor record and fetches the correct worklist.
- Added guards so the UI does not crash if the doctors API returns an unexpected shape during initial auth timing.
- Fixed the patient panel link flow and imported the missing `Link` dependency.
- Updated the appointments response to include patient `gender`, which was required by the doctor-side patient details panel.

Result

- The doctor dashboard now shows seeded appointments correctly.
- The patient details panel opens correctly.
- The history-records page is reachable and functional.

Optimizations Performed

- Replaced N+1 fetching with relation includes.
- Parallelized independent queries using `Promise.all`.
- Moved pagination and filtering into database queries.
- Reduced avoidable frontend crashes and rerender-related instability.
- Improved response handling consistency between frontend and backend.

Known Issues / Remaining Work

1. Architectural issue: Doctor and User are not properly linked

The application still has a schema-level design gap:

- `User` and `Doctor` are separate models.
- There is no formal foreign key relation such as `Doctor.userId -> User.id`.

Because of that, I applied a safe fallback in the frontend so the doctor dashboard works now. However, the proper long-term fix would be:

- add `userId` to the `Doctor` model
- define a Prisma relation between `Doctor` and `User`
- update seed logic and authentication-dependent flows accordingly

2. Queue concurrency issue

The queue check-in flow still contains a concurrency/race-condition vulnerability by design in the original assignment. A stronger production fix would require:

- transactional token generation
- locking or atomic sequencing strategy
- possibly a unique daily token constraint per doctor

3. Deployment

Deployment is still pending. The expected plan is:

- frontend on Vercel
- backend on Render
- database on Neon or another hosted PostgreSQL provider

4. Submission artifacts still pending

- deployed application URL
- video walkthrough
- final Google Form submission

Reasoning Behind Major Decisions

Prioritizing security first

Security and authorization issues were fixed first because they create the highest real-world risk. Password leakage, weak token handling, SQL injection, and broken admin authorization are more critical than UI polish.

Using Prisma ORM wherever possible

I preferred Prisma-native querying over manual SQL because it improved safety, readability, and maintainability while also reducing injection risk.

Fixing user-facing crashes early

Frontend crashes directly affect the reviewer’s ability to navigate the app. Fixing null access bugs, hook ordering issues, and broken response handling made the application much easier to test and demonstrate.

Completing the missing feature

The missing history page was completed because incomplete features are directly called out in the assignment and are easy for reviewers to verify.

Using a pragmatic workaround for the doctor mapping issue

Since the doctor dashboard was broken by a schema mismatch, I implemented a safe functional fix to restore the workflow without blocking the rest of the assignment. I would still document the missing database relation as a known technical debt item rather than pretend it is fully solved at the data-model level.

Final Summary

This submission improves the HAQMS project across:

- security
- authorization
- backend correctness
- query performance
- frontend stability
- missing feature completion
- doctor workflow reliability

The core coding/debugging portion of the assignment is substantially complete. The main remaining work is deployment, final documentation sharing, video recording, and submission.